certora

Security Assessment

Final Report

Kamino yvaults

August 2025

Prepared for Kamino Finance

Table of content

Project Summary 3
Project Scope 3
Project Overview 3
Findings Summary 5
Severity Matrix 5
Detailed Findings 6
High Severity Issues 7
H-01 Unprivileged scope feed initialization function 7
Medium Severity Issues 8
M-01 Deposit validation bypass in amount calculation 9
Low Severity Issues 10
L-01 Silent error suppression in price retrieval 10
L-02 Missing validation for proportional deposits 11
L-03 Redundant Account Initialization in SignTerms 12
L-04 Incorrect Boundary Condition in Range Validation 13
L-05 Array bounds mismatch in reward index check 14
Informational Severity Issues 15
I-01. Documentation-implementation mismatch in reward mapping logic 15
I-02. Confusing variable naming in flash instruction validation 15
I-03. Outdated struct naming in strategy definition 16
I-04. Magic number in reward array definition 17
I-05. Raw integer type used for rebalance configuration 17
I-06. Poor readability in rebalance type comparison 17
I-07. Commented code left in production 18
I-08. Error name mismatch in position validation 19
I-09. Inaccurate comment in account validation 19
I-10. Inconsistent naming in withdrawal effects 20
I-11. Unclear parameter naming in reward initialization 20
I-12. Ineffective type conversion in reward initialization 21
I-13. Confusing function name in reward validation 22
I-14. Confusing function name in reward account validation 22
I-15. Incomplete memory cleanup in rebalance state update 23
I-16. Typo in flash_swap_uneven_vault.rs 24
I-17. flash_swap_uneven_vaults_end can be called independently 24
Disclaimer 25
About Certora 25

Project Summary

Project Scope

Project Name	Repository (link)	Latest Commit Hash	Platform
yvaults	github.com/Kamino-Finance/yvaults	187b060	Solana

Fixes and additional changes in the following PRs:

PR	Repository (link)	Latest Commit Hash	Platform
#988	github.com/Kamino-Finance/yvaults	686746e	Solana
#985	github.com/Kamino-Finance/yvaults	a6df1d7	Solana
#956	github.com/Kamino-Finance/yvaults	39abd7f	Solana

Project Overview

This document describes the security assessment of **Kamino yvaults** using **manual code review**. The work was undertaken from August 4th, 2025 to August 22nd 2025.

The audit was primarily focussed on the following instructions to verify if the program upgrade to support multiple scope feeds introduces any security issues:

```
open_liquidity_position()
deposit()
withdraw()
single_token_deposit_with_min()
flash_swap_uneven_vaults_start()
flash_swap_uneven_vaults_end()
```

While focussing on these instructions, any issues that were found in other parts of the codebase are also included in this report.

Protocol Overview

Kamino's yvaults is an automated liquidity management tool designed for Concentrated Liquidity Market Makers (CLMMs) on Solana. It supports different strategies to manage positions on Orca, Raydium and Meteora.
When users deposit into a strategy, the protocol seamlessly handles liquidity placement, reward optimization, and maintenance. By automating complex actions—swapping, compounding, and rebalancing—it allows users to earn consistent yields with minimal manual effort.

The YVaults program uses scope prices to ensure accurate and reliable pricing data is used for its operations. Scope prices are utilized to validate token values, calculate liquidity positions, and manage rebalancing strategies effectively.
By integrating scope price feeds, the program maintains consistency in price updates, enabling precise execution of actions such as deposits, withdrawals, and rebalancing. The program has been updated to support multiple scope feeds.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	0	0	0
High	1	1	1
Medium	1	1	1
Low	5	4	4
Informational	17	14	14
Total	24	20	20

Severity Matrix

Impact	Critical	Medium	Medium/High	High	Critical
	High	Low/Medium	Medium	Medium/High	High
	Medium	Low	Low/Medium	Medium	Medium/High
	Low	Informational	Low	Low/Medium	Medium
		Rare	Unlikely	Likely	Very Likely
		Likelihood			

Detailed Findings

ID	Title	Severity	Status
H-01	Unprivileged scope feed initialization function	High	Fixed
M-01	Deposit validation bypass in amount calculation	Medium	Fixed
L-01	Silent error suppression in price retrieval	Low	Fixed
L-02	Missing validation for proportional deposits	Low	Fixed
L-03	Redundant Account Initialization in SignTerms	Low	Acknowledged
L-04	Incorrect Boundary Condition in Range Validation	Low	Fixed
L-05	Array bounds mismatch in reward index check	Low	Fixed

High Severity Issues

H-01 Unprivileged scope feed initialization function		
Severity: **High**	Impact: **High**	Likelihood: **Likely**
Files: lib.rs	Status: Fixed	

Description: The token_infos_init_scope_feed instruction is unprivileged and allows any user to overwrite the scope_feed field in all token infos. This function was designed as a temporary initialization tool to populate the scope_feed field after it was added in a recent update.

The intended deployment sequence is: deploy the update, call this function once using the single allowed scope feed from global_config, then remove the function and redeploy. However, if the function remains deployed when a second scope feed is added to the global config, any user can call it to overwrite existing scope feeds with different values.

This creates a denial-of-service vector where malicious actors can corrupt the scope feed configuration, potentially causing strategies that depend on specific price feeds to fail or behave incorrectly.

Recommendations: Implement strict operational policies and processes to ensure this temporary initialization function is removed before allowing additional scope feeds in the configuration. The protocol must establish mandatory procedures that prevent the addition of a second scope feed until this function has been completely removed from the deployed program.

Alternative security controls include:

- Adding a privileged signer requirement to restrict function access
- Implementing validation that only allows updates when the previous scope_feed value is set to default
- Creating automated deployment checks that verify function removal before configuration updates

The safest approach remains establishing rigorous operational controls to guarantee removal of this temporary function once all token infos have been properly configured and before any expansion of scope feed capabilities.

Customer's response: Addressed using a specific wallet only

Fix Review: Fix confirmed

Medium Severity Issues

M-01 Deposit validation bypass in amount calculation		
Severity: **Medium**	Impact: **Medium**	Likelihood: **Likely**
Files: vault_operations.rs	Status: Fixed	

Description: The deposit function performs minimum deposit validation using assert_deposit_value_higher_than_minimum_allowed before calculating the actual depositable amounts. This validation uses the user's maximum input amounts (max_a_amount, max_b_amount) rather than the final amounts that will actually be deposited.

The issue occurs because get_depositable_amounts is called after the validation, which can significantly reduce the deposit amounts based on the strategy's current state and minting method. This means a user could pass the minimum deposit check with large input amounts but end up depositing much smaller amounts that fall below the intended minimum threshold.

For example, if a user provides amounts worth $1000 (passing validation) but the proportional deposit logic only allows $50 worth to maintain proper ratios, the actual deposit bypasses the minimum deposit protection.

Recommendations: Move the assert_deposit_value_higher_than_minimum_allowed call to after get_depositable_amounts to validate the actual amounts that will be deposited rather than the user's maximum input amounts.

Customer's response: Addressed

Fix Review: Fix confirmed

Low Severity Issues

L-01 Silent error suppression in price retrieval		
Severity: **Low**	Impact: **Low**	Likelihood: **Likely**
Files: scope.rs	Status: Fixed	

Description: The price retrieval mechanism that is used in multiple places in the code uses a problematic pattern that silently suppresses critical pricing errors. When fetching USD prices for strategy tokens, the code uses .ok() to convert all pricing failures into None values, effectively discarding valuable error context.
Downstream code may encounter unexpected None values where valid prices are expected, potentially leading to panics when these values are unwrapped or used in calculations.

Example:

```rust
#scope.rs:526
let price_a = get_price_usd(
        &all_scope_prices.token_a,
        &token_infos[token_a.to_usize()],
        slot,
    )
    .ok(); // <--- error is ignored, returns None

let mut prices = Box::<StrategyTokenPrices>::default();
prices.token_a_price = price_a;

//-------------------------
#vault_operations.rs:259
let token_a_price = prices.token_a_price.unwrap(); // panic when price is None
```

Recommendations: Either propagate the price error or add checks to handle None values for the price in all places where the price fields are used.

Customer's response: Addressed

Fix Review: Fix confirmed

L-02 Missing validation for proportional deposits

Severity: **Low**	Impact: **Low**	Likelihood: L**ikely**
Files: vault_operations.rs	Status: Fixed	

Description: The deposit function includes a check for uninvested value limits using assert_uninvested_value_less_than_maximum_allowed(&holdings) but only applies it to price-based deposits. This validation ensures that strategies don't accumulate excessive uninvested funds that could indicate rebalancing issues.

However, this same check is bypassed for proportional deposits, even though proportional deposits can also contribute to uninvested value accumulation. The check is currently positioned within the price-based deposit branch.
This means proportional deposits can potentially exceed the maximum acceptable available value threshold without triggering the safety check.

This affects the consistency of deposit validation across different minting methods. Proportional deposits could accumulate uninvested funds beyond intended limits without proper validation.

Recommendations: Move the assert_uninvested_value_less_than_maximum_allowed check outside the price-based deposit conditional block so it applies to both minting methods, ensuring consistent validation regardless of the deposit type.

Customer's response: Intentional, applying this for proportional would block correct deposits.

Fix Review: Acknowledged

L-03 Redundant Account Initialization in SignTerms		
Severity: **Low**	Impact: **Low**	Likelihood: L**ikely**
Files: sign_terms.rs	Status: Acknowledged	

Description: The sign_terms instruction contains a redundant account initialization pattern that will lead to initialization failures. The instruction uses Anchor's init constraint to create and initialize an account, but then attempts to load the same account using load_init() method.

Recommendations: Replace the load_init() call with load_mut() to correctly access the already-initialized account.

Customer's response: Not an issue

L-04 Incorrect Boundary Condition in Range Validation

Severity: **Low**	Impact: **Low**	Likelihood: **Likely**
Files: vault_operations.rs	Status: Fixed	

Description: The assert_strategy_in_range function contains an incorrect boundary condition that uses a strict inequality when validating the lower price bound. The current implementation checks whether lower_tick_price < current_price, which excludes the scenario where the current price exactly equals the lower tick price.

In concentrated liquidity market maker protocols, positions are typically valid when the current price falls within the defined range boundaries, including when the price sits exactly at the boundary ticks. The exclusion of the boundary condition creates an unnecessary restriction that could prevent legitimate operations when the market price precisely aligns with the strategy's lower tick.

Recommendations: Modify the boundary condition to use an inclusive comparison for the lower bound, changing the validation to check lower_tick_price <= current_price. This adjustment ensures that positions remain valid when the current price exactly equals the lower tick price.

Customer's response: Addressed

Fix Review: Fix confirmed

L-05 Array bounds mismatch in reward index check

Severity: **Low**	Impact: **Low**	Likelihood: **Low**
Files: update_rewards_mapping.rs	Status: Fixed	

Description: The validation logic in update_rewards_mapping correctly limits reward_index to values that are valid for Orca and Raydium protocols, both of which support reward indices up to the upper bound defined by DEX_REWARDS_UPPER_INDEX. However, Meteora's LbPair.reward_infos array has a smaller capacity, supporting only indices 0 and 1, while the global upper index allows values up to 2.

When a caller provides a reward_index of 2 for a Meteora strategy, the input passes the initial validation check since 2 is within the global upper bound. However, when the code reaches the Meteora-specific branch and attempts to access pool.reward_infos[reward_index] with index 2, it triggers an array bounds violation that results in a runtime panic.

Recommendations: Update the check to also give a VaultError::InvalidRewardIndex error instead of a panic in the case of reward_index 2 for a Meteora strategy.

Customer's response: Addressed

Informational Severity Issues

I-01. Documentation-implementation mismatch in reward mapping logic

File: update_reward_mapping.rs
Description: The update_reward_mapping instruction contains a significant inconsistency between its documented behavior and actual implementation in the reward mapping update logic. The code comment at line 77 states that when a reward mapping is already set, the function should "ignore the update," suggesting that existing mappings should be preserved without error.

However, the actual implementation directly contradicts this documented behavior. When the code detects that a reward vault has already been configured (indicated by the vault address differing from the base_vault_authority), it immediately returns an error using VaultError::RewardVaultOverrideNotAllowed rather than silently ignoring the update attempt.

Recommendations: The comment should be updated to accurately reflect the actual implementation behavior, clarifying that the function returns an error when attempting to override existing reward vault configurations. Alternatively, if the original "ignore" behavior is the intended design, the implementation should be modified to silently succeed when encountering already-configured reward mappings, ensuring consistency between documentation and actual functionality.

Customer's response: Addressed

I-02. Confusing variable naming in flash instruction validation

File: Location: flash_ixs.rs
Description: In the search_second_ix function, a variable is named first_ix but actually loads the instruction at index 1.
When validating flash loan transactions with compute budget instructions, the code checks that

instructions at indices 0 and 1 are both compute budget instructions when the current instruction is at index 2.

The variable first_ix at line 113 loads instruction_at(1), which is the second instruction in the transaction, not the first. This naming doesn't match what the variable contains.

Recommendations: Rename the variable to second_ix to accurately reflect that it contains the instruction at index 1.

Customer's response: Addressed

I-03. Outdated struct naming in strategy definition

File: state.rs
Description: The main strategy struct is named WhirlpoolStrategy but the program now supports multiple DEX protocols beyond just Whirlpool/Orca. The struct is used for strategies across Orca, Raydium, and Meteora protocols, making the current name specific to only one of the supported DEXes.

The struct contains a strategy_dex field that determines which protocol the strategy uses, confirming that this is a generic strategy structure rather than one specific to Whirlpool.

Recommendations: Rename the struct to a more generic name like Strategy or CLMMStrategy while maintaining the same discriminator to ensure compatibility with existing on-chain accounts.

Customer's response: Acknowledged

I-04. Magic number in reward array definition

File: state.rs
Description: The WhirlpoolStrategy struct defines the kamino_rewards field using a hardcoded array size of 3 instead of referencing the KAMINO_REWARDS_COUNT constant that exists elsewhere in the codebase.

Using magic numbers makes the code harder to maintain and creates potential inconsistencies if the reward count needs to be changed in the future. The codebase already has a dedicated constant for this value that should be used consistently across all reward-related array definitions.

Recommendations: Replace the hardcoded 3 with the KAMINO_REWARDS_COUNT constant in the array definition: pub kamino_rewards: [KaminoRewardInfo; KAMINO_REWARDS_COUNT].

Customer's response: Addressed

I-05. Raw integer type used for rebalance configuration

File: state.rs
Description: The WhirlpoolStrategy struct defines the rebalance_type field as a raw u8 instead of using a proper enum type.

Recommendations: Replace the u8 type with a proper RebalanceType enum that defines the valid rebalancing modes. This would provide compile-time validation and make the code more self-documenting about supported rebalancing strategies.

Customer's response: Acknowledged

I-06. Poor readability in rebalance type comparison

File: open_liquidity_position.rs
Description: The constraint validation for manual rebalancing uses a numeric comparison strategy.load()?.rebalance_type > 0 instead of a more explicit enum comparison. This creates a

dependency on the underlying numeric values of the RebalanceType enum and makes the code less self-documenting.

The current implementation assumes that RebalanceType::Manual has a value of 0 and all other rebalance types have values greater than 0. While this may be correct, it obscures the actual intent of the check, which is to determine whether the rebalance type is manual or not.

Recommendations: Replace the numeric comparison with an explicit enum comparison: strategy.load()?.rebalance_type != RebalanceType::Manual. This makes the intent clear and provides better type safety regardless of the underlying enum values.

Customer's response: Addressed

I-07. Commented code left in production

File: lib.rs
Description: The deposit function contains a large block of commented-out code that includes feature compilation checks and an operation restriction. The commented section shows:

```rust
// if cfg!(not(feature = "integration_tests")) { @audit remove comments
//     return Err(VaultError::OperationForbidden.into());
// }
```

This appears to be leftover code from when the deposit function was restricted to integration tests only. The commented logic would have prevented the function from being used in production environments, but this restriction has since been removed.

Recommendations: Remove the commented-out code block. If this compilation logic needs to be preserved for reference, it should be documented in version control history rather than kept as comments in the active codebase.

Customer's response: Addressed

I-08. Error name mismatch in position validation

File: open_liquidity_position.rs
Description: The error handling logic for position opening contains a naming inconsistency where the error name doesn't accurately describe the condition being checked. When a strategy has status Active or Frozen, the code returns VaultError::StrategyNotActiveWhenOpeningPosition.

The error name suggests the issue is that the strategy is not in an active state, when in reality the problem is the opposite - the strategy is already active and has an existing position that would conflict with opening a new one.

Recommendations: Rename the error to better reflect the actual condition, such as VaultError::CannotOpenPositionWithExistingPosition to accurately describe that the error occurs when trying to open a position while one already exists.

Customer's response: Addressed

I-09. Inaccurate comment in account validation

File: open_liquidity_position.rs
Description: The token_infos account in the OpenLiquidityPosition struct has a comment stating "strategy checks this" but the actual validation is performed by the global_config account. The account constraint shows #[account(has_one = token_infos)] on the global_config field, indicating that global_config is responsible for validating the token_infos account relationship.

Recommendations: Update the comment to accurately reflect that global_config validates the token_infos account: /// CHECK: global_config checks this.

Customer's response: Addressed

I-10. Inconsistent naming in withdrawal effects

File: executive_withdraw.rs
Description: The handler_executive_withdraw function uses EmergencyWithdrawalEffects to destructure the result of the executive_withdraw operation, but the struct name doesn't match the context. The function performs executive withdrawal operations, not emergency withdrawals, creating a naming mismatch between the operation type and the effects struct.

This inconsistency can confuse developers about whether emergency and executive withdrawals use the same underlying logic or if there are differences between these operations that should be reflected in the naming.

Recommendations: Rename the struct from EmergencyWithdrawalEffects to ExecutiveWithdrawalEffects to match the context in which it's being used.

Customer's response: Addressed

I-11. Unclear parameter naming in reward initialization

File: initialize_kamino_reward.rs
Description: The handler_initialize_kamino_reward function has a parameter named collateral_token that represents an index or identifier value rather than an actual token object. The parameter is used as an array index to access token_infos[id] and is converted to a CollateralToken enum, indicating it's a numeric identifier rather than token data.

The current name collateral_token suggests the parameter contains token information, when it actually contains a reference value used to look up token information from other data structures. This naming creates confusion about the parameter's purpose and data type.

Recommendations: Rename the parameter to collateral_token_index or collateral_token_id to clearly indicate that it represents a lookup value rather than the token data itself. This would make the function signature more self-documenting and align with how the parameter is actually used in the implementation.

Customer's response: Addressed

I-12. Ineffective type conversion in reward initialization

File: initialize_kamino_reward.rs
Description: The handler_initialize_kamino_reward() function converts a u64 parameter to CollateralToken using CollateralToken::try_from(collateral_token) but only uses the converted value for logging purposes. The CollateralToken is essentially a wrapper around u64 that doesn't add any extra validation - the try_from conversion always succeeds.

All subsequent operations continue to use the original collateral_token parameter rather than the converted token value for array indexing and function calls.

The conversion only affects the logging output format:

With CollateralToken: "collateral token: CollateralToken(2)"
With raw u64: "collateral token: 2"
The same logging output could be achieved without the conversion by using: "Initialising kamino rewards with id: {:?} and collateral token: CollateralToken({:?})" with the raw collateral_token parameter.

Recommendations: Either use the converted token variable consistently throughout the function, or remove the conversion and format the logging message directly to include the type name if desired.

Customer's response: Addressed

I-13. Confusing function name in reward validation

File: initialize_kamino_reward.rs
Description: The utils::assert_permissions function name doesn't match its actual purpose. The function checks two things:

1. That kamino_reward_index is within the valid range (<= DEX_REWARDS_UPPER_INDEX)
2. That the reward at that index is not already initialized (!strategy.kamino_rewards[index].initialized())

Neither check relates to user permissions or access control. The function validates reward availability and prevents duplicate initialization rather than checking permissions.

Recommendations: Rename the function to reflect its actual purpose, such as validate_reward_availability or check_reward_not_initialized. This would make the function's behavior clear and help developers understand what validation is being performed.

Customer's response: Addressed

I-14. Confusing function name in reward account validation

File: add_kamino_rewards.rs
Description: The utils::assert_permissions function name doesn't match its actual purpose. The function performs three account validation checks:

1. Verifies that the reward at kamino_reward_index is initialized
2. Confirms that ctx.accounts.reward_vault.key() matches the strategy's stored reward vault
3. Confirms that ctx.accounts.reward_mint.key() matches the strategy's stored reward mint

These checks validate that the provided accounts match the expected reward configuration rather than checking user permissions or access control.

Recommendations: Rename the function to reflect its actual purpose, such as validate_reward_accounts or verify_reward_configuration. This would make it clear that the function validates account relationships rather than checking permissions.

Customer's response: Addressed

I-15. Incomplete memory cleanup in rebalance state update

File: vault_operations.rs
Description: The rebalance function updates the strategy's state by copying new state bytes but doesn't clear any remaining bytes if the new state is shorter than the previous one:

```rust
 strategy.rebalance_raw.state[..rebalance_step.new_state_bytes.len()]
.copy_from_slice(&rebalance_step.new_state_bytes);
```

When new_state_bytes is shorter than the existing rebalance_raw.state, the leftover bytes from the previous state remain in memory. While the current implementation doesn't cause issues because deserialization ignores bytes beyond the expected length, it's good practice to keep state data clean.

Recommendations: Add memory cleanup to zero out unused bytes after updating the state:

Customer's response: Addressed

I-16. Typo in flash_swap_uneven_vault.rs

File: flash_swap_uneven_vault.rs:35

Description: Change from **current_ix_progrm_id** to **current_ix_program_id**.

Customer's response: Addressed

I-17. flash_swap_uneven_vaults_end can be called independently

File: programs/yvaults/src/instructions/flash_swap_uneven_vaults.rs

Description: `**handler_end**` can be invoked without a matching start which is misleading. In practice it can only move funds user→vault or revert, but it increases confusion and integration error risk.

Recommendations: Disallow calling "end" standalone.

Customer's response: Acknowledged

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.